

Unit IV:

Managing Exceptions, Applet Programming:

Introduction to Errors and Exceptions

- In programming, errors are Unavoidable. They can arise from various sources, such as incorrect input, logical flaws in the code, or unexpected system conditions. When an error occurs during program execution, it can lead to unexpected behavior, program crashes, or incorrect results. To prevent these issues, programmers use error and exception handling techniques.
- **Error:** An error indicates a problem that prevents the program from executing correctly.
- **Exception:** An exception is a specific type of error that occurs during the execution of a program, disrupting the normal flow of instructions.

Types of Errors

Errors can be broadly classified into three main categories:

- **Syntax Errors:** These errors occur when the code violates the grammatical rules of the programming language. They are usually detected by the compiler or interpreter before the program runs. Examples include:

- Missing semicolons
- Unmatched parentheses or brackets
- Incorrect keywords
- Undeclared variables

- **Logical Errors:** These errors occur when the code compiles and runs without producing syntax errors, but the program does not behave as intended due to flaws in the program's logic. These are the hardest to find. Examples include:

- Incorrect formulas or algorithms
- Using the wrong comparison operator
- Infinite loops
- Off-by-one errors

- **Runtime Errors:** These errors occur during the execution of the program. They are often caused by unexpected conditions or invalid operations. Examples include:
 - Division by zero
 - Accessing an array element out of bounds
 - Trying to open a file that does not exist
 - Running out of memory

Exceptions

- **Exception Handling** is a mechanism in Java to **handle runtime errors** (like dividing by zero, accessing an invalid array index, etc.) so that the **normal flow of the program is maintained**.
- Without exception handling, your program may crash when an error occurs.

Syntax of Exception Handling Code

- Most programming languages provide a try-catch block to handle exceptions. The basic syntax is as follows:

```
try {  
    // Code that might throw an exception  
} catch (ExceptionType1 e1) {  
    // Code to handle ExceptionType1  
} catch (ExceptionType2 e2) {  
    // Code to handle ExceptionType2  
} finally {  
    // Code that always executes, regardless of whether an  
    exception occurred  
}
```

- **try:** The try block encloses the code that might throw an exception.
- **catch:** The catch block specifies the type of exception to catch and the code to execute when that exception occurs.
- **finally:** The finally block contains code that always executes, regardless of whether an exception occurred or not. It is typically used to release resources, such as closing files or database connections.

Keyword**Purpose**

try

Contains code that might throw an exception.

catch

Used to handle the exception.

finally

Contains code that always executes (cleanup code).

throw

Used to throw an exception manually.

throws

Declares exceptions that a method might throw.

```
public class Example1 {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0; // risky code  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by  
zero!");  
        }  
        System.out.println("Program continues...");  
    }  
}
```

Multiple Catch Statements

- A try block can be followed by multiple catch blocks to handle different types of exceptions. The catch blocks are examined in order, and the first catch block that matches the type of the exception is executed.

syntax

```
try {  
    // Code that might throw an exception  
} catch (ArithmeticException e) {  
    // Handle arithmetic exceptions  
} catch (ArrayIndexOutOfBoundsException e) {  
    // Handle array index out of bounds exceptions  
} catch (Exception e) {  
    // Handle any other exceptions  
}
```

```
public class MultipleCatchExample {  
    public static void main(String[] args) {  
        try {  
            int[] arr = {1, 2, 3};  
            System.out.println(arr[5]); // ArrayIndexOutOfBoundsException  
            int num = 10 / 0;          // ArithmeticException  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array index out of range!");  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero!");  
        }  
        catch (Exception e) {  
            System.out.println("Some other exception occurred!");  
        }  
    }  
}
```

Using Finally Statement

- The finally block is a part of Java's **exception handling mechanism** (try-catch-finally).
- It is used to execute **important cleanup code** — code that must run **whether or not an exception occurs**.
- The finally block **always executes** — even if an exception is thrown,

Syntax

```
try {  
    // Code that may throw an exception  
}  
catch (ExceptionType e) {  
    // Code to handle the exception  
}  
finally {  
    // Code that always executes  
}
```

```
public class FinallyExample {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0; // Risky code (will throw exception)  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Exception caught: Cannot divide  
by zero!");  
        } finally {  
            System.out.println("Finally block is always executed.");  
        }  
  
        System.out.println("Program continues...");  
    }  
}
```

Throwing Our Own Exceptions

- Sometimes, you want to signal that something has gone wrong — not because of a Java system error (like divide-by-zero), but because of a **logical condition** in your program.
- In such cases, you can **manually throw an exception** using the throw keyword.

```
public class ThrowExample {  
    public static void main(String[] args) {  
        int age = 15;  
  
        if (age < 18) {  
            // throwing an exception manually  
            throw new ArithmeticException("Not eligible to vote");  
        } else {  
            System.out.println("Eligible to vote");  
        }  
  
        System.out.println("End of program");  
    }  
}
```

Using Exceptions for Debugging

- **Debugging** means finding and fixing errors in your program.

Java's **exception-handling system** provides a powerful built-in way to detect, locate, and understand runtime errors

- Exceptions can be valuable tools for debugging.
- When an exception occurs, it provides information about the type of error, the location in the code where the error occurred, and the call stack.
- This information can help programmers identify and fix the root cause of the error

Example

```
public class DebugExample {  
    public static void main(String[] args) {  
        int[] numbers = {10, 20, 30};  
  
        // This will cause an exception (index 5 does  
        not exist)  
        System.out.println("Number: " + numbers[5]);  
  
        System.out.println("End of program");  
    }  
}
```

- Java throws an **ArrayIndexOutOfBoundsException**.
- The program stops and shows **error details**, including **line number**, which helps us **debug**.
- **Exception in thread "main"**
java.lang.ArrayIndexOutOfBoundsException:
Index 5 out of bounds for length 3
at DebugExample.main(DebugExample.java:6)

Introduction to Applets

What is an Applet?

- An applet is a small Java program that runs inside a web browser or an applet viewer. It is used mainly for creating interactive web applications, animations, and graphics.
- Applets are part of Java's GUI (Graphical User Interface) programming

Key Features of Applets

- Applets run **inside a browser or viewer**, not directly from the command line.
- They **don't have a main()** method — instead, they use special **lifecycle methods**.
- Applets are **secure** (they cannot access local files or system resources).
- They can be **embedded** in HTML pages using the `<applet>` or `<object>` tag.
- They are **event-driven** — respond to user actions like mouse clicks, keystrokes, etc.

How Applets Differ from Applications

Feature	Applet	Application
Meaning	Small Java program that runs inside a web browser or applet viewer.	Standalone Java program that runs directly on the computer (JVM).
Main Method	No main() method — uses init(), start(), stop(), destroy().	Has a main() method — program starts from there.
Execution	Runs inside a browser or applet viewer.	Runs from the command line or an IDE.
Run Command	appletviewer filename.html	java ClassName
Input/Output	Uses GUI (graphics, text fields, buttons) for input/output.	Can use console input/output (System.out.println, etc.).

Security	Runs in a restricted (sandbox) environment — limited access to system.	Full access to local system resources (files, databases, etc.).
GUI	Runs inside a browser window.	Runs in its own window (Frame).
Life Cycle	Controlled by browser — uses methods like init(), start(), etc.	Controlled by programmer — uses main() function.
HTML	Embedded inside HTML using <applet> tag.	No HTML needed.
Usage	For small, interactive web programs.	For desktop or console applications.

Preparing to Write Applets

❖ Requirements:

- Java Development Kit (JDK) installed.
- Text editor (e.g., Notepad, VS Code, etc.).
- Applet Viewer or a Java-enabled web browser to execute the applet.

❖ Important Package

import the Abstract Window Toolkit (AWT) and Applet packages:

```
import java.applet.*;
```

```
import java.awt.*;
```

❖ Rules for Writing an Applet:

- Applet class must extend the Applet class.
- Applets do not have a main() method.
- Display output using the paint(Graphics g) method.
- The applet must be embedded in an HTML file or run with Applet Viewer.

Building Applet Code

Basic Structure of an Applet:

```
import java.applet.*;
```

```
import java.awt.*;
```

```
/*
```

```
<applet code="MyFirstApplet.class" width="300" height="150">
```

```
</applet>
```

```
*/
```

```
public class MyFirstApplet extends Applet {
```

```
    public void paint(Graphics g) {
```

```
        g.drawString("Hello, Welcome to Applet Programming!", 20, 20);
```

```
    }
```

```
}
```

Create an HTML file (e.g., applet.html) with:

```
<html>
```

```
<body>
```

```
<applet code="MyFirstApplet.class"  
width="300" height="150"></applet>
```

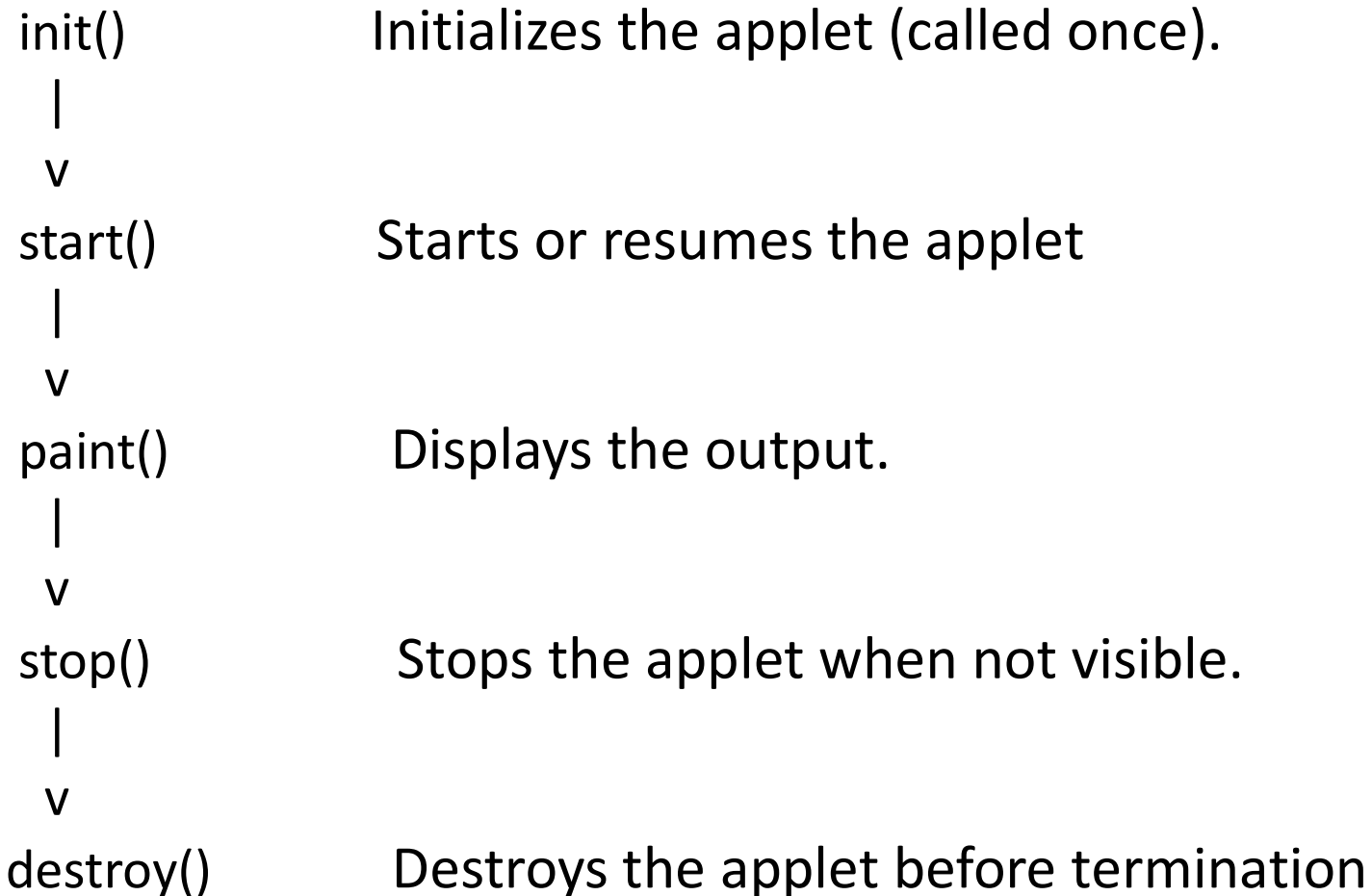
```
</body>
```

```
</html>
```


Applet Life Cycle

Stage	Method	Description
1. Initialization	init()	Called once when the applet is first loaded. Used for initialization (like setting colors, loading images, etc.).
2. Starting	start()	Called each time the applet becomes active (when browser window is opened or revisited).
3. Running / Painting	paint(Graphics g)	Called automatically whenever the applet needs to display output on the screen.
4. Stopping	stop()	Called when the applet is no longer active (when user leaves the page or minimizes the window).
5. Destroying	destroy()	Called once when the applet is about to be destroyed, used to release resources (memory, files, etc.).

Applet Life Cycle Diagram



Example Showing All Life Cycle Methods

```
import java.applet.*;  
import java.awt.*;
```

```
/*  
<applet code="LifeCycleDemo.class" width="400" height="200">  
</applet>  
*/
```

```
public class LifeCycleDemo extends Applet {  
  
    public void init() {  
        System.out.println("Applet is initialized.");  
    }  
}
```

```
}
```

```
public void start() {  
    System.out.println("Applet is started.");  
}
```

```
public void paint(Graphics g) {  
    g.drawString("Applet Life Cycle Example", 50, 100);  
}
```

```
public void stop() {  
    System.out.println("Applet is stopped.");  
}
```

```
public void destroy() {  
    System.out.println("Applet is destroyed.");  
}  
}
```

Design a Web Page using an Applet

```
import java.applet.*;
```

```
import java.awt.*;
```

```
public class MyApplet extends Applet {
```

```
    public void paint(Graphics g) {
```

```
        g.drawString("Welcome to My Web Page!",  
50, 50);
```

```
    }
```

```
}
```

```
<html>
  <head>
    <title>My Applet Page</title>
  </head>
  <body>
    <applet code="MyApplet.class" width="300"
height="200">
      </applet>
  </body>
</html>
```

<APPLET> Tag

Definition:

- The <applet> tag in HTML is used to embed a Java Applet inside a web page.
- It tells the browser which applet class file to load and how to display it

Basic Syntax:

```
<applet code="AppletClassName.class"  
width="width" height="height">  
</applet>
```

Example:

```
<applet code="MyApplet.class" width="300"  
height="200">  
</applet>
```


Adding Applet to an HTML File

1. Write the Applet program in Java and save it as **.java** file.

Example: **MyApplet.java**

Compile the file:

javac MyApplet.java

This creates **MyApplet.class**.

Create an HTML file

```
<html>
```

```
  <body>
```

```
    <applet code="MyApplet.class" width="300"  
height="200">
```

```
    </applet>
```

```
  </body>
```

```
</html>
```

Running the Applet

(a) Using Applet Viewer

Applet Viewer is a command-line tool that runs applets outside a browser

```
appletviewer MyAppletPage.html
```

b) Using a Web Browser

Open the HTML file (**MyAppletPage.html**) in a browser that supports Java applets

More About <APPLET> Tag

```
<applet code="ExampleApplet.class"  
        width="400" height="200"  
        codebase="classes/"  
        archive="applet.jar"  
        alt="Applet cannot be displayed."  
        name="exampleApplet"  
        align="middle"  
        hspace="10" vspace="10">  
</applet>
```

Description of tags :

- **codebase** → Path of the directory where class file is located.
- **archive** → Used when the applet code is inside a .jar file.
- **alt** → Displayed if applet cannot load.
- **hspace/vspace** → Adds space around the applet.
- **align** → Aligns the applet (left, right, top, bottom, middle).

Using <param> Tag with Applet

- The <param> tag is used **inside the <applet> tag** in the HTML file.
- It allows passing **name–value pairs** as parameters to the Java applet.
- Inside the applet, these parameters are retrieved using the **getParameter()** method
- The <param> tag passes **parameters (values)** from HTML to the Java applet.

Syntax:

```
<param name="parameter_name"  
value="parameter_value">
```

```
import java.applet.*;
import java.awt.*;
/*
<applet code="ParamDemo.class" width="300" height="150">
    <param name="studentName" value="John">
    <param name="marks" value="85">
</applet>
*/
```

```
public class ParamDemo extends Applet {
    String name;
    String marks;

    public void init() {
        // Get the parameter values from HTML file
        name = getParameter("studentName");
        marks = getParameter("marks");
    }
}
```

```
// Check if parameters are missing
```

```
    if (name == null)
```

```
        name = "Unknown";
```

```
    if (marks == null)
```

```
        marks = "0";
```

```
}
```

```
public void paint(Graphics g) {
```

```
    g.drawString("Student Name: " + name, 50, 60);
```

```
    g.drawString("Marks: " + marks, 50, 80);
```

```
}
```

```
}
```



```
<html>
<head>
  <title>Passing Parameters to Applet</title>
</head>
<body>
  <h2>Example: Passing Parameters to Applet</h2>
  <applet code="ParamDemo.class" width="300"
height="150">
    <param name="studentName" value="John">
    <param name="marks" value="85">
  </applet>
</body>
</html>
```

Aligning the Display

When we display text, images, or drawings inside an **applet**, we often want to **align** them properly — for example, in the center, left, or right side of the applet window.

- When embedding an applet in an HTML page, you can control its **alignment on the web page** using the **align** attribute of the `<applet>` tag.

Syntax:

```
<applet code="MyApplet.class" width="300"  
height="150"  
align="alignment_value"></applet>
```

```
<html>
```

```
<body>
```

```
  <h3>Aligning the Applet Example</h3>
```

```
  <p>This applet is aligned to the left:</p>
```

```
  <applet code="AlignApplet.class"  
width="200" height="100"  
align="left"></applet>
```

```
  <p>This text appears beside the applet.</p>
```

```
</body>
```

```
</html>
```

Common Values for align

Value	Meaning
left	Aligns the applet to the left side of the web page.
right	Aligns the applet to the right side of the web page.
top	Aligns the applet to the top relative to surrounding text or elements.
bottom	Aligns the applet to the bottom relative to other content.
middle	Centers the applet vertically with surrounding text.

More About HTML Tags

While applets are primarily Java code, they exist within an HTML environment. You can use standard HTML tags around the `<applet>` tag to provide context, formatting, and structure to your web page.

- **Text Formatting:** Use tags like `<p>`, `<h1>` to `<h6>`, `<b>`, `<i>`, `<u>` to format text around the applet.
- **Lists:** Use `<ul>`, `<ol>`, and `<li>` to create lists.
- **Tables:** Use `<table>`, `<tr>`, `<td>`, and `<th>` to create tables.
- **Links:** Use `<a>` to create hyperlinks.

Displaying Numerical Values

Applets can display numerical values using the `drawString()` method of the `Graphics` class. You'll need to convert the numerical value to a string before displaying it.

Example:

```
import java.applet.Applet;  
import java.awt.Graphics;  
  
public class NumberApplet extends Applet {  
    int number = 123;  
    double pi = 3.14159;  
    |  
    public void paint(Graphics g) {  
        g.drawString("Integer: " + Integer.toString(number), 20, 30);  
        g.drawString("Double: " + Double.toString(pi), 20, 60);  
    }  
}
```


Getting Input from the User

- Applets can get input from the user using various AWT (Abstract Window Toolkit) components, such as:
- **TextField:** Allows the user to enter a single line of text.
- **TextArea:** Allows the user to enter multiple lines of text.
- **Button:** Triggers an action when clicked.
- **Checkbox:** Allows the user to select an option.
- **Choice:** Provides a dropdown list of options.

Example

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

/*
<applet code="SimpleApplet" width=300 height=150>
</applet>
*/

public class SimpleApplet extends Applet implements ActionListener {
    TextField t;
    Button b;
    String msg = "";

    public void init() {
        // Create components
        t = new TextField(15);
        b = new Button("Submit");

        // Add to applet
        add(new Label("Enter your name:"));
        add(t);
        add(b);
    }
}
```

```
// Add action listener to button
    b.addActionListener(this);
}
```

```
public void actionPerformed(ActionEvent e) {
    msg = "Hello, " + t.getText() + "!";
    repaint();
}
```

```
public void paint(Graphics g) {
    g.drawString(msg, 50, 100);
}
}
```

```
<html>
```

```
<body>
```

```
<applet code="SimpleApplet.class" width="300"  
height="150"></applet>
```

```
</body>
```

```
</html>
```

Introduction to Graphics Programming

- Graphics programming is used to create **visual output** like shapes, images, and animations.
- In Java, graphics are mainly handled using the **AWT (Abstract Window Toolkit)** library.
- The main class for graphics is the **Graphics** class.
- Every component (like an Applet or Frame) has a paint() method where drawing is done.

The Graphics Class

- **Purpose:** The Graphics class in Java is used for drawing shapes, text, and images on components like **Applets** or **Frames**.
- **Package:** java.awt.*

Commonly Used Methods:

Method	Description
<code>drawLine(int x1, int y1, int x2, int y2)</code>	Draws a straight line from (x1, y1) to (x2, y2)
<code>drawRect(int x, int y, int width, int height)</code>	Draws rectangle outline with top-left corner at (x, y)
<code>fillRect(int x, int y, int width, int height)</code>	Draws filled rectangle
<code>setColor(Color c)</code>	Sets the color of the drawing
<code>drawString(String s, int x, int y)</code>	Draws text at specified coordinates

1.Drawing Lines

Lines are drawn using the **drawLine()** method.

Syntax:

```
g.drawLine(x1, y1, x2, y2);
```

(x1, y1) → starting point

(x2, y2) → ending point


```
import java.applet.Applet;
import java.awt.*;

/*
<applet code="LineApplet" width=400 height=300>
</applet>
*/

public class LineApplet extends Applet {
    public void paint(Graphics g) {
        g.setColor(Color.BLUE);
        g.drawLine(50, 50, 200, 50); // horizontal line
        g.setColor(Color.RED);
        g.drawLine(50, 50, 50, 150); // vertical line
        g.setColor(Color.GREEN);
        g.drawLine(50, 50, 200, 150); // diagonal line
    }
}
```

```
<html>
<head>
  <title>Line Applet</title>
</head>
<body>
  <h2>Line Drawing Applet</h2>

  <applet code="LineApplet.class" width="400"
height="300">
    Your browser does not support Java Applets.
  </applet>

</body>
</html>
```

Drawing rectangle

```
g.drawRect(x, y, width, height);
```

Where:

- x → x-coordinate of top-left corner
- y → y-coordinate of top-left corner
- width → width of the rectangle
- height → height of the rectangle

```
import java.applet.Applet;  
import java.awt.*;
```

```
/*  
<applet code="RectangleApplet" width=300 height=200>  
</applet>  
*/
```

```
public class RectangleApplet extends Applet {  
    public void paint(Graphics g) {  
        g.setColor(Color.BLUE);  
        g.drawRect(50, 50, 150, 80); // x, y, width, height  
    }  
}
```

```
<html>
<head>
  <title>Rectangle Applet</title>
</head>
<body>

  <applet code="RectangleApplet.class" width="300"
height="200">
</applet>

</body>
</html>
```

circles and ellipses

Java provides two main methods for oval shapes:

✓ **drawOval(int x, int y, int width, int height)**

Draws only the **outline** of the circle / ellipse.

```
import java.applet.Applet;
```

```
import java.awt.*;
```

```
public class SimpleOvalApplet extends Applet {
```

```
    public void paint(Graphics g) {
```

```
        // Draw a simple circle
```

```
        g.drawOval(50, 50, 100, 100); // width = height → circle
```

```
        // Draw a simple ellipse
```

```
        g.drawOval(200, 50, 150, 80); // width ≠ height → ellipse
```

```
    }
```

```
}
```

```
<html>
```

```
<body>
```

```
<applet code="SimpleOvalApplet.class"  
width="400" height="200">
```

```
</applet>
```

```
</body>
```

```
</html>
```


Drawing Arcs in Java Applet

```
g.drawArc(x, y, width, height, startAngle, arcAngle);
```

Parameter	Meaning
x, y	Top-left corner of the bounding rectangle
width, height	Size of the oval from which the arc is taken
startAngle	Starting position of the arc (in degrees)
arcAngle	How many degrees the arc will sweep

```
import java.applet.Applet;
import java.awt.*;

public class ArcApplet extends Applet {
    public void paint(Graphics g) {

        // Simple arc (open arc)
        g.drawArc(50, 50, 150, 100, 0, 180);
        // Starts at 0°, draws half circle (180°)

        // Filled arc (pie shape)
        g.fillArc(250, 50, 150, 100, 0, 90);
        // 0° to 90° → quarter circle
    }
}
```

```
<html>
```

```
<body>
```

```
<applet code="ArcApplet.class"  
width="500" height="200">
```

```
</applet>
```

```
</body>
```

```
</html>
```

Drawing Polygons in Java Applet

Java provides two ways to draw polygons:

`drawPolygon(int x[], int y[], int n)`

```
import java.applet.Applet;
import java.awt.*;

public class PolygonApplet extends Applet {

    public void paint(Graphics g) {

        int x[] = {50, 150, 100};
        int y[] = {150, 150, 50};

        g.drawPolygon(x, y, 3); // 3 points = triangle
    }
}
```

```
<html>
<body>
  <applet
code="PolygonApplet.class"
width="400" height="300">
  </applet>
</body>
</html>
```

Line Graphs

- **What is a Line Graph?**
- A line graph is a chart that uses straight line segments to connect data points. It is commonly used to show trends over time.
- A line graph typically contains:
 - Axes (X-axis and Y-axis)
 - Data Points
 - Lines connecting data points

`g.drawLine(x1, y1, x2, y2)` → draws a straight line

```
import java.applet.Applet;
import java.awt.Graphics;

public class LineGraphApplet extends Applet {

    public void paint(Graphics g) {

        // Draw Axes
        g.drawLine(50, 250, 350, 250); // X-axis
        g.drawLine(50, 250, 50, 50); // Y-axis

        // Sample data points
        int[] data = {30, 60, 45, 80, 90};

        int x = 60; // starting x position
        int prevX = x;
        int prevY = 250 - data[0];

        // First point
        g.fillOval(prevX - 2, prevY - 2, 4, 4);
```



```
// Draw lines between points
    for (int i = 1; i < data.length; i++) {
        x += 50;
        int y = 250 - data[i];

        g.fillOval(x - 2, y - 2, 4, 4); // draw point
        g.drawLine(prevX, prevY, x, y); // connect previous to
current

        prevX = x;
        prevY = y;
    }
}
```

```
<html>
  <body>
    <h2>Simple Line Graph Applet</h2>

    <applet code="LineGraphApplet.class"
width="400" height="300">
      </applet>

  </body>
</html>
```

Using Control Loops in Applets

- Control Loops are used to repeat a block of code multiple times.
- In Java Applets, loops are often used to draw multiple shapes, patterns, or charts dynamically instead of writing repeated code.

Loop Type

Description

for

Repeats code a **known number of times**

while

Repeats code **while a condition is true**

do-while

Executes code **at least once** and then
repeats while condition is true

```
import java.applet.Applet;  
import java.awt.*;
```

```
public class LoopApplet extends Applet {
```

```
    public void paint(Graphics g) {
```

```
        // Using a for loop to draw 5 circles  
        for(int i = 0; i < 5; i++) {
```

```
            g.drawOval(20 + (40 * i), 50, 30, 30);
```

```
        }
```

```
    }
```

```
}
```

```
<html>
```

```
<body>
```

```
<applet code="LoopApplet.class" width="300" height="200">
```

```
</applet>
```

```
</body>
```

```
</html>
```

Drawing Bar Charts

```
import java.applet.Applet;
import java.awt.Graphics;
import java.awt.Color;

public class BarChartApplet extends Applet {
    public void paint(Graphics g) {
        int[] quantities = {10, 15, 7};
        String[] fruits = {"Apples", "Bananas", "Oranges"};

        int x = 50;
        int barWidth = 50;
```

```
for(int i = 0; i < fruits.length; i++) {  
    int barHeight = quantities[i] * 10;  
    g.setColor(Color.BLUE);  
    g.fillRect(x, 200 - barHeight, barWidth, barHeight);  
    g.setColor(Color.BLACK);  
    g.drawRect(x, 200 - barHeight, barWidth,  
barHeight);  
    g.drawString(fruits[i], x, 220);  
    x += 80;  
}  
g.drawString("Fruits Sold in a Store", 80, 20);  
}  
}
```

```
<html>
```

```
<body>
```

```
<applet code=" BarChartApplet.class"  
width="300" height="200">
```

```
</applet>
```

```
</body>
```

```
</html>
```